

Pressure Sensor Initial Testing

Author: Brendan Klem

Goal: Build an Arduino circuit that can output pressure sensor data.

Materials List

- Arduino Uno R3
- Resistors: 10kΩ
- 3 Wires
- Pressure sensor
- USB-C Cable

Pressure Sensor Differences

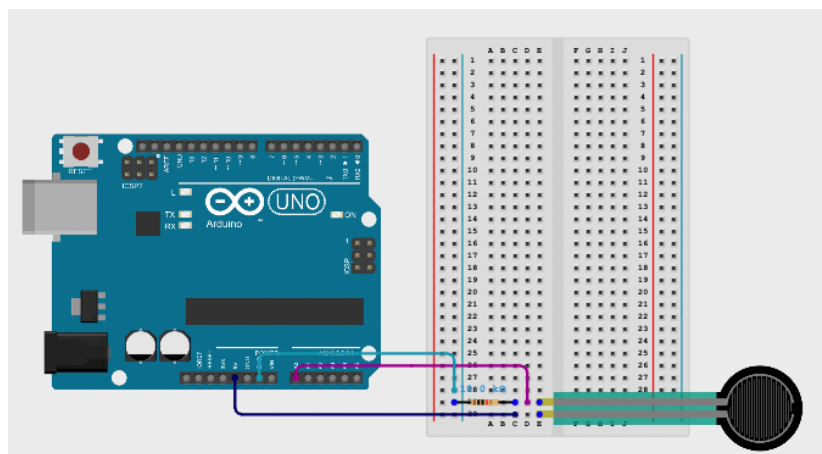
I found that the two different types of sensors that you sent have some pros and cons.

- TE pressure sensor (Smaller one): I used a 100kΩ resistor as listed on the data sheet but the values were extremely small, only showing numbers from 0-3, so I had to move it up to a 1MΩ resistor in order to increase the data range.
 - Pros: The sensor itself is smaller so could be easier to fit in small spaces.
 - Cons: The values were a bit inconsistent compared to the other pressure sensor, and sometimes the values would randomly cut back to 0.
- Flexiforce sensor (Larger one): I used a 10kΩ resistor as listed on the data sheet and it seemed to work pretty well to get a large range of values.
 - Pros: Values are very consistent and correspond quickly to changes in pressure.
 - Cons: The sensor is large so it could be more difficult to fit in small spaces.

I also found there is a conductive material called velostat that can be used as a pressure sensor, and is very flexible. We have some on order and I plan on testing it once it comes in.

Circuit Layout:

Here is the simple circuit layout I used to test the pressure sensors. A more complex circuit may be required as we increase the scope.



Testing Code

Code used to test the pressure values:

```
1  int sensorPin = A0;
2
3  void setup() {
4      Serial.begin(115200);
5  }
6
7  void loop() {
8      int value = analogRead(sensorPin);
9      Serial.println(value);
10     delay(100);
11 }
```

Sample Output

TE Sensor Output:

When I applied a hard amount of pressure, the values jumped up to around 118 but there are some issues with the sensor randomly returning 0 values into the output as seen in the beginning of the trial.

```
21:51:54.827 -> 0
21:51:54.925 -> 0
21:51:55.024 -> 0
21:51:55.123 -> 0
21:51:55.223 -> 0
21:51:55.323 -> 0
21:51:55.422 -> 1
21:51:55.521 -> 2
21:51:55.620 -> 2
21:51:55.722 -> 5
21:51:55.821 -> 2
21:51:55.920 -> 1
21:51:56.019 -> 89
21:51:56.117 -> 100
21:51:56.216 -> 95
21:51:56.315 -> 108
21:51:56.414 -> 114
21:51:56.513 -> 112
21:51:56.612 -> 118
21:51:56.711 -> 118
21:51:56.810 -> 127
21:51:56.909 -> 120
```

Flexiforce sensor Output:

The starting values are very consistent at 8, and when pressure is applied the values have a wide range, varying from about 8-220 from my testing.

```
22:02:05.445 -> 10
22:02:05.544 -> 8
22:02:05.644 -> 7
22:02:05.741 -> 24
22:02:05.839 -> 59
22:02:05.938 -> 84
22:02:06.036 -> 90
22:02:06.136 -> 86
22:02:06.235 -> 93
22:02:06.332 -> 117
22:02:06.431 -> 127
22:02:06.531 -> 113
22:02:06.631 -> 125
22:02:06.729 -> 121
22:02:06.828 -> 129
22:02:06.923 -> 124
22:02:07.022 -> 120
22:02:07.153 -> 127
22:02:07.252 -> 110
22:02:07.349 -> 19
22:02:07.449 -> 8
```

Saving the Data to a File

To save the data from the pressure sensor, there are a few approaches that could work:

Option 1: MQTT over Wi-Fi

The Arduino can send the data wirelessly to another device, where it gets saved

real-time data collection and doesn't require a wired connection to a computer.

Option 2: SD Card Storage

The Arduino can write the sensor data directly to a file on an SD card. This lets it store data without needing a computer or internet connection. The downside to this is that there is no real-time data and it can only be accessed after the experiment is complete.

Option 3: Python via Serial

The Arduino sends data through the serial port to a computer, and a Python script reads it and saves it to a file. This is the easiest method and works well for local testing, but this would require a wired connection to a computer.